

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



WEB PROGRAMMING

GROUP PROJECT *VIETTALENT JOB VACANCY MANAGEMENT*

Instructor: Dr. Nguyễn Đức Thái

Students:

Nguyễn Văn Viết	2353316
Nguyễn Khắc Thái Khang	2352481
Tổng Hữu Đạt	2352244
Nguyễn Lê Cao Tường	2353300

Group: 6

HO CHI MINH CITY, APRIL 2026



Team Contributions

Member	Student ID	Phase	Main Contribution
Nguyễn Văn Viết	2353316	Phase 1	Built the project foundation, folder structure, database schema, seed data, MVC-like core structure, authentication, session handling, role-based access control, and base layout.
Tổng Hữu Đạt	2352244	Phase 2	Implemented employer features, including employer dashboard, structured job vacancy creation, required-skills dynamic form, job editing, deletion, activation/deactivation, and ownership protection.
Nguyễn Lê Cao Tướng	2353300	Phase 3	Implemented public and job-seeker features, including homepage, job listing, keyword search, multi-criteria filtering, AND-logic query building, sorting, job detail view, and responsive public UI.
Nguyễn Khắc Thái Khang	2352481	Phase 4	Implemented administrator features, including admin dashboard, job vacancy moderation, lookup and location data management, final integration, testing checklist, README, report support, and demo preparation.

Contents

Team Contributions	i
1 Requirement and Source Code Consistency Review	1
2 Introduction	4
2.1 Project Background	4
2.2 Project Goals	4
2.3 Scope and Out-of-Scope Items	5
2.4 Document Structure	5
3 System Roles and Functional Requirements	6
3.1 Roles	6
3.2 Functional Requirement Mapping	6
3.3 Assessment Criteria Alignment	8
3.4 Use Cases at a Glance	8
4 System Architecture	9
4.1 Architectural Style	9
4.2 Folder Layout	10
4.3 Request Lifecycle	10
4.4 Technology Stack	11
5 Database Design	12
5.1 Conceptual Model	12
5.2 Design Decisions and Their Rationale	12
5.3 Naming Conventions	13
6 Entity-Relationship Diagram	14
6.1 Cardinality Catalogue	15
6.2 Key Tables Explained	17
users	17
employer_profiles	17



	job_vacancies	17
	job_vacancy_skills	17
	Lookup tables	18
	Location tables	18
7	Normalization Analysis	18
7.1	First Normal Form (1NF)	18
7.2	Second Normal Form (2NF)	19
7.3	Third Normal Form (3NF)	19
7.4	Boyce–Codd Normal Form (BCNF)	20
7.5	Why a Junction Table for Skills?	20
7.6	Indexes	20
8	Job Search Implementation	21
8.1	Filter Vocabulary	21
8.2	AND-Logic Combination	21
8.3	Sorting	22
8.4	Job Detail View	23
8.5	Performance Notes	23
9	Security and Validation	23
9.1	Authentication	23
9.2	Role-Based Access Control	23
9.3	Ownership Protection	24
9.4	Server-Side Validation	24
9.5	Database Hardening	25
9.6	Output Encoding	25
9.7	CSRF Considerations	25
10	User Interface and Experience	26
10.1	Layout System	26
10.2	Responsive Design	26
10.3	Dynamic Forms	26
10.4	Accessibility	27
10.5	Internationalisation	27
11	Testing and Demo	27
11.1	Test Strategy	27
11.2	Manual Test Checklist	27



11.3	Demo Script	28
11.4	Known Edge Cases	28
12	Limitations and Future Work	29
12.1	Deliberate Out-of-Scope Items	29
12.2	Limitations of the Current Implementation	29
12.3	Future Work	30
13	Conclusion	30
1	Appendix: SQL Schema File	31
2	Appendix: Default Demo Accounts	31
3	Appendix: Main Routes	32
4	Appendix: Website Setup and Run Instructions	32
5	Appendix: Build Instructions	33



1 Requirement and Source Code Consistency Review

Before moving the content into the HCMUT template, the report was checked against the assignment specification, the provided folder-structure guideline, and the available VietTalent source-code evidence. The overall result is that the report is consistent with the current project scope and implementation. The most important checks are summarized in Table 1.1.

Table 1.1: Requirement coverage audit.

Requirement area	Evidence in report / source code	Status
Employer authentication and role guard	The report describes register, login, logout and role-based routing. The route table and controller flow use protected employer/admin routes.	Covered
Employer job creation and management	Vacancy creation, edit, view, delete, activate/deactivate and ownership checks are documented. The employer controller stores vacancies through structured foreign-key fields and required-skill rows.	Covered
Structured job location	Location is modeled with <code>countries</code> , <code>cities</code> , and <code>districts</code> ; <code>job_vacancies</code> stores <code>country_id</code> , <code>city_id</code> , and optional <code>district_id</code> , not one free-text location.	Covered



Requirement area	Evidence in report / source code	Status
Reference tables for selectable fields	The report and schema use lookup tables for job titles, categories, industries, employment types, job levels, salary ranges, salary types, skills, proficiencies, degree levels, experience levels, and work arrangements.	Covered
Maximum five required skills	The report states the cap and includes it in manual testing. The skill relationship is stored in <code>job_vacancy_skills</code> .	Covered; verify manually in UI before demo
Many-to-many vacancy-skill relationship	The ERD and schema include <code>job_vacancy_skills(job_vacancy_id, skill_id, proficiency_level_id)</code> with a composite primary key.	Covered
Job seeker search, filtering, sorting, and detail view	The search section explains keyword, category, location, skill, employment type, level, salary range, work arrangement, and sorting options using AND logic.	Covered
Administrator management	The report covers admin job status control, reference data management, and location management.	Covered



Requirement area	Evidence in report / source code	Status
MVC or equivalent architecture	The architecture section follows the required structure: <code>public/index.php</code> as front controller, <code>app/</code> for controllers/models/core, <code>config/</code> for database settings, and <code>resources/views/</code> for templates.	Covered
Responsive UI and usability	The UI/UX section documents responsive layout, reusable layout elements, form feedback, and accessibility considerations.	Covered
Out-of-scope constraints	The report explicitly excludes CV creation/search, job application workflow, and matching/recommendation algorithms.	Covered
Deliverables	The report includes ERD, SQL schema, architecture explanation, testing checklist, demo script, and route appendix.	Covered

Important limitation of this review. The uploaded files contained the report source and template. For source-code verification, I also cross-checked the public GitHub repository information that matches the project name and path. This report should still be rechecked once more against the exact local final submission folder immediately before uploading, because local files may differ from the public repository or from earlier generated report content.



2 Introduction

2.1 Project Background

This report documents the design and implementation of **VietTalent**, a web application that handles two complementary halves of the recruitment process: (i) employers create and manage structured job vacancy postings; (ii) job seekers and visitors browse, search, filter and inspect those postings. The project is developed for Assignment 2 of the Web Programming module, which extends Assignment 1 (Online CV Management) by reusing the same user roles and the same controlled vocabularies, but *focusing on the employer side of recruitment*. CV creation, CV search and the application workflow are explicitly out of scope.

2.2 Project Goals

The system pursues four concrete goals:

- **Structured data first.** Every selectable job attribute (title, category, industry, employment type, level, salary range, salary type, work arrangement, country, city, district, degree level, experience level, skill, proficiency) is stored as a foreign key into a reference table rather than free text. This is required by the specification and is the dominant design constraint.
- **Normalized location.** The job location is decomposed into `country_id`, `city_id`, optional `district_id` and `work_arrangement_id` – never a single string.
- **Many-to-many required skills.** Required skills are modelled through a junction table `job_vacancy_skills` that also records the minimum proficiency level. Storing skills as JSON or as a comma-separated string is forbidden by the specification.
- **Multi-criteria search with AND-logic.** Job seekers can combine keyword, category, location, skill, employment type, level, salary range and work arrangement filters; the result must satisfy all of them simultaneously.



2.3 Scope and Out-of-Scope Items

The scope deliberately follows the assignment specification. The features in Table 2.1 are implemented, while the features in the right-hand column are explicitly excluded.

Table 2.1: Scope summary.

Area	In scope	Out of scope
Authentication	Register, login, logout, role guard	Email verification, OAuth, password reset emails
Employer	Vacancy CRUD on own postings, dynamic skill picker, activate/deactivate	Vacancy templates, bulk import
Job seeker	Browse, keyword search, multi-criteria filters, sorting, detail view	Apply for jobs, save jobs, alerts, recommendations
Administrator	Manage all vacancies, manage lookup and location data, soft-remove postings	Multi-tenant, audit log UI
Database	Normalized relational schema with FKs and a junction table	Search engine, full-text indexes

2.4 Document Structure

Section 3 maps the three user roles and the functional requirements directly onto features in the running system. Section 4 describes the MVC architecture, the folder layout, and the request lifecycle. Section 5 introduces the conceptual data model, while Section 6 embeds the rendered ER diagram and walks through every relationship. Section 7 contains a normalization analysis up to BCNF and explains the design decisions that produced the final schema. Section 8 explains the search query strategy and AND-logic combination. Sections 9–12 cover security, UI, testing and limitations. The appendix gives the complete DDL.



3 System Roles and Functional Requirements

3.1 Roles

The system has three roles, encoded in `users.role` as the SQL ENUM (`'admin'`, `'employer'`, `'job_seeker'`). Role-based access control is enforced inside controllers via `Auth::requireRole()`.

- **Employer** – creates and manages *their own* job vacancies. An employer cannot view or modify another employer's postings; the controller layer rejects such requests on ownership mismatch.
- **Job seeker / visitor** – read-only access to public job pages. Browsing is allowed without login; logged-in seekers see the same pages.
- **Administrator** – moderation and reference-data steward. The administrator can soft-remove inappropriate vacancies (`status='removed'`) and maintain every lookup table including the structured location tables.

3.2 Functional Requirement Mapping

Table 3.1 maps the specification's numbered requirements onto the implemented controllers, models and routes.



Table 3.1: Specification requirement to implementation mapping.

Spec §	Requirement	Implementation
4.1.1	Employer auth, RBAC	<code>AuthController</code> , <code>Auth::requireRole('employer')</code>
4.1.2A	Basic job info (title, category, type, industry, level, openings)	<code>job_vacancies</code> FKs to 5 lookup tables, integer column for openings
4.1.2B	Structured location (country, city, district, arrangement)	<code>countries</code> / <code>cities</code> / <code>districts</code> cascade and <code>work_arrangements</code> FK
4.1.2C	Salary range, salary type, benefits	<code>salary_ranges</code> , <code>salary_types</code> , <code>benefits</code> TEXT
4.1.2D	Job description (free text)	5 TEXT columns: <code>responsibilities</code> , <code>required_qualifications</code> , <code>preferred_skills</code> , <code>benefits</code> , <code>additional_notes</code>
4.1.2E	Required skills (max 5) with proficiency	<code>job_vacancy_skills</code> junction with <code>proficiency_level_id</code> ; client + server cap at 5
4.1.2F	Min degree level, min experience	<code>degree_levels</code> , <code>experience_levels</code> FKs
4.1.3	Employer vacancy CRUD + activate/deactivate	<code>EmployerController</code> , owner check on every action, <code>status</code> ENUM
4.2.1	Job-seeker multi-criteria search	<code>PublicJobController</code> , dynamic WHERE clause builder
4.2.2	AND-logic combinable filters	All filters appended as AND predicates with bound pa- rameters
4.2.3	Sorting (recent, salary asc/desc, alpha)	Whitelisted <code>ORDER BY</code> col- umn resolution
4.2.4	Job detail view	<code>job_detail.php</code> view with one query joining all look up tables



3.3 Assessment Criteria Alignment

Table 3.2 summarises how the implemented system aligns with the assignment grading criteria. The highest-weighted items are job vacancy management, public search/filtering, and database normalisation, so the report gives these areas the most detailed treatment.

Table 3.2: Assessment criteria alignment.

Assessment category	Weight	Coverage in VietTalent
Job vacancy creation and management	30%	Structured employer form, vacancy CRUD, activation/deactivation, required-skill picker, and ownership protection.
Job search and filtering	25%	Public search page, keyword search, lookup filters, AND-logic combination, skill EXISTS filter, sorting, and detail page.
Database design and normalisation	20%	Relational schema, lookup/reference tables, structured location, salary ranges, and many-to-many vacancy-skill relationship.
Code structure and architecture	15%	MVC-like PHP structure with controllers, models, views, core helpers, validation, and role guards.
UI and usability	5%	CareerViet-inspired public pages, dashboard layouts, reusable components, validation feedback, and responsive CSS.
Documentation and demo	5%	Technical report, ERD, SQL schema, README, manual test checklist, and demo walkthrough.

3.4 Use Cases at a Glance

The most important workflows are summarised below. They all execute through the front controller `public/index.php`, which dispatches to controllers based on the `?page=` query parameter.

Employer creates a vacancy. After login, the employer opens `?page=employer_job_create`. The controller fetches every active lookup row (titles, categories, industries, employment

types, job levels, salary ranges, salary types, degree levels, experience levels, work arrangements, countries, skills, proficiency levels) and renders the form. Cascading selects for city and district are populated client-side via fetch calls scoped to the chosen `country_id` / `city_id`. On submit, server-side validation re-resolves every FK against the database and rejects any unknown id.

Job seeker searches. A visitor opens `?page=jobs` and applies any subset of: keyword, category, country, city, work arrangement, employment type, job level, salary range, required skill. The controller assembles a parameterised query that joins `job_vacancies` with the relevant lookup tables, applies one `WHERE` predicate per active filter (combined with `AND`), filters `status='active'`, and orders by the user's chosen sort key.

Administrator removes a posting. The admin opens `?page=admin_jobs`, filters or paginates the list, and changes the status to `removed`. The vacancy is no longer surfaced on public pages, but the row – and its skill links – are preserved for audit purposes (no hard delete).

4 System Architecture

4.1 Architectural Style

The system follows a classical **Model–View–Controller (MVC)** architecture, implemented from scratch in PHP without a third-party framework. The choice avoids vendor lock-in and keeps the code transparent for academic review, while still demonstrating the same patterns one would find in Laravel, Symfony or CodeIgniter.

- **Models** (`app/Models/`) encapsulate every SQL query. Each model exposes intention-revealing methods (`JobVacancy::search()`, `Lookup::active()`, `User::findByEmail()`) and returns associative arrays.
- **Controllers** (`app/Controllers/`) coordinate between input, validation, models, and views. They enforce role-based access control and ownership rules before mutating data.
- **Views** (`resources/views/`) are PHP templates with embedded HTML, organised by audience: public, auth, employer, admin, partial and layout.



- **Core** (`app/Core/`) contains four cross-cutting utilities – **Database**, **Router**, **Controller** (base class), **Auth** and **Validator** – so each concrete controller stays small.

4.2 Folder Layout

The repository follows the conventional “framework-style” layout that separates source code, configuration, public web root and resources.

Folder	Purpose
<code>app/Core/</code>	Database, Router, base Controller, Auth, Validator
<code>app/Controllers/</code>	7 controllers: Home, Auth, Employer, Admin, Dashboard, Language, PublicJob
<code>app/Models/</code>	User, EmployerProfile, JobVacancy, JobVacancySkill, Lookup
<code>app/Helpers/</code>	URL, session and translation helpers
<code>config/</code>	<code>app.php</code> and <code>database.php</code> (loaded once by the front controller)
<code>database/</code>	<code>schema.sql</code> – 19 tables; <code>seed.sql</code> – demo accounts and sample lookups
<code>public/</code>	<code>index.php</code> (front controller), <code>css/</code> , <code>js/</code> – the <i>only</i> web-accessible folder
<code>resources/views/</code>	Layouts, partials, public, auth, employer, admin and error templates
<code>storage/</code>	Reserved for uploads / cache
<code>docs/</code>	Database notes, demo script, test checklist

The deployment under XAMPP places the project at `C:\xampp\htdocs\VietTalent_JobVacancyManagement`. The application is reached at `http://localhost/VietTalent_JobVacancyManagement/public/`, which keeps every non-public file out of the web root.

4.3 Request Lifecycle

The lifecycle from HTTP request to rendered HTML is straightforward and easy to reason about:



1. Apache forwards `public/index.php`.
2. The front controller boots `config/app.php` and `config/database.php`, registers the autoloader and starts the session.
3. `Router::dispatch()` reads `?page=` and resolves it to a `Controller::action` pair.
4. The controller checks authentication and role with `Auth::requireRole()`, validates input via `Validator`, and calls one or more models.
5. Models execute prepared statements via `Database::pdo()` (a PDO singleton in `utf8mb4` with `ATTR_ERRMODE = ERRMODE_EXCEPTION`).
6. The controller calls `view()` with the template path and a context array. The base `Controller` extracts variables and includes the layout, which in turn includes the chosen view.

This contained, predictable cycle is what makes server-side validation, role guards and ownership protection cheap to implement consistently.

4.4 Technology Stack

The stack is intentionally conservative – the assignment is graded on data modelling and clean PHP, not on framework cleverness.

Layer	Technology
Web server	Apache 2.4 (XAMPP)
Server-side runtime	PHP 8.x with PDO_MySQL
Database	MySQL 8.x, character set <code>utf8mb4</code>
Frontend	HTML5, vanilla CSS, vanilla JavaScript (no SPA framework)
Source control	Git (commit history per phase)
Local environment	XAMPP for Windows



5 Database Design

5.1 Conceptual Model

At the conceptual level the schema has **five families of entities**, summarised below. The full inventory is 19 tables.

- **Identity entities.** `users` carry credentials and a role; `employer_profiles` extends users with company information; `job_seeker_profiles` extends users with seeker-side metadata. Both profile tables hold a **UNIQUE** foreign key to `users.id`, which encodes a one-to-one specialisation rather than a strict inheritance hierarchy.
- **Central operational entity.** `job_vacancies` is the heart of the system. Every selectable attribute on the job creation form is recorded here as a foreign key.
- **Junction entity for skills.** `job_vacancy_skills` connects `job_vacancies` to `skills` and additionally records the minimum proficiency level for that skill on that posting.
- **Reference (lookup) entities.** 12 tables – `job_titles`, `job_categories`, `industries`, `employment_types`, `job_levels`, `salary_ranges`, `salary_types`, `degree_levels`, `experience_levels`, `work_arrangements`, `skills`, `proficiency_levels` – each modelled as (`id`, `name|label`, `status`).
- **Location entities.** `countries` → `cities` → `districts`, a 3-level hierarchy with **UNIQUE** on (`country_id`, `name`) and (`city_id`, `name`).

5.2 Design Decisions and Their Rationale

The schema was shaped by five top-level choices, each driven by a specific clause of the assignment specification.

1. Foreign keys instead of free text for *every* selectable field. The specification explicitly forbids storing job requirements or location as plain text or JSON. Beyond compliance, this gives consistent select option labels, validation at the database boundary, and pre-computable filters for the search page.



2. Hierarchical location split over three tables. A single `location` string would make “filter by country” or “filter by city” practically impossible. With `countries / cities / districts` the search controller can join the chain and apply equality predicates on `country_id` or `city_id` cheaply. `district_id` is intentionally nullable because not every employer posts at district granularity.

3. Salary as a reference range *plus* a numeric column for sorting. `salary_ranges` stores both a human label (e.g. “1000–1500 USD”) and the numeric `min_salary / max_salary` columns. The label is what the employer picks; `min_salary` is what the public search uses for `ORDER BY`. This keeps the user-facing vocabulary controlled while still enabling correct numeric sort.

4. Skills as a many-to-many junction table. The required-skills set is itself an attribute of the relationship between a vacancy and a skill. `job_vacancy_skills` carries the additional `proficiency_level_id`, which is a property of the relationship and not of the skill itself. The spec caps the relationship at five skills per vacancy; this cap is enforced in PHP rather than in SQL because MySQL has no row-count constraint, and the application is the only writer of this table.

5. Soft delete via status for vacancies. Hard-deleting a posting would lose history. Vacancy status is therefore an ENUM (`'active'`, `'inactive'`, `'removed'`); the public search filter is `status='active'`, the employer dashboard hides `removed`, and the admin moderation action sets the status to `removed`.

5.3 Naming Conventions

The schema uses lowercase plural table names, snake_case columns, `id` for surrogate primary keys, and `<table>_id` for foreign keys. Every foreign key has an explicit `CONSTRAINT fk_*` name to make migration diffs readable. Every lookup table has a single `UNIQUE` business key (`name` or `label`) so seed scripts are idempotent. Status columns are ENUMs with a short, closed set of values rather than free text.

6 Entity–Relationship Diagram

The schema is rendered in Figure 6.1. Reference tables are grouped together, the central `job_vacancies` table is highlighted in dark red, the user/profile cluster is in blue, the skill junction in purple, and the structured location chain in orange. Crow’s-foot notation is used: the “many” side carries the crow’s foot, the “one” side carries a single bar.

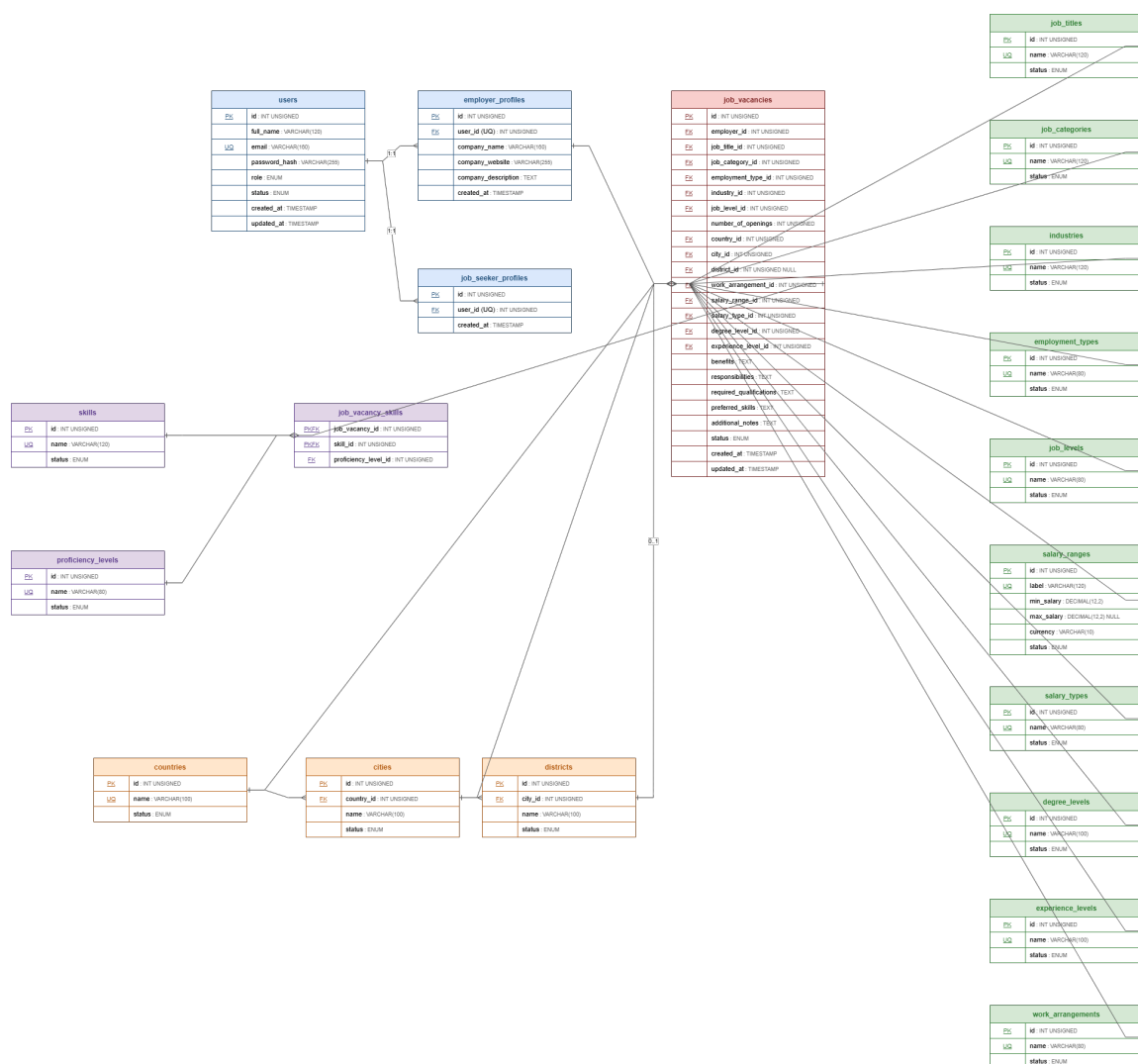


Figure 6.1: Entity–Relationship diagram of the VietTalent schema.



6.1 Cardinality Catalogue

Table 6.1 enumerates every relationship in the diagram together with its cardinality and whether the child side is mandatory.



Table 6.1: Relationship catalogue.

Parent (1)	Child (N)	Mandatory?	Notes
users	employer_profiles	yes (1:1)	Unique <code>user_id</code>
users	job_seeker_profiles	yes (1:1)	Unique <code>user_id</code>
employer_profiles	job_vacancies	yes	Owner of the posting
job_titles	job_vacancies	yes	Title is a reference value
job_categories	job_vacancies	yes	
employment_types	job_vacancies	yes	Full-time, Part-time, Internship, Contract
industries	job_vacancies	yes	
job_levels	job_vacancies	yes	Junior, Mid, Senior
salary_ranges	job_vacancies	yes	Drives display + sort
salary_types	job_vacancies	yes	Gross / Net
degree_levels	job_vacancies	yes	Minimum required degree
experience_levels	job_vacancies	yes	Minimum years of experience
work_arrangements	job_vacancies	yes	Onsite / Remote / Hybrid
countries	job_vacancies	yes	
cities	job_vacancies	yes	
districts	job_vacancies	no (0..1)	Nullable – not every job needs a district
countries	cities	yes	Hierarchical chain
cities	districts	yes	Hierarchical chain
job_vacancies	job_vacancy_skills	yes	ON DELETE CASCADE
skills	job_vacancy_skills	yes	
proficiency_levels	job_vacancy_skills	yes	Minimum proficiency for that skill



6.2 Key Tables Explained

users

The single source of truth for authentication. Three columns are noteworthy:

- **role** is an ENUM, not a foreign key, because the set of roles is closed and the application code switches on it directly in `Auth::requireRole()`.
- **status** (active/inactive) lets administrators temporarily disable an account without deleting it.
- **password_hash** is filled by `password_hash($plain, PASSWORD_DEFAULT)` (bcrypt). Plaintext passwords never reach the database.

employer_profiles

A one-to-one specialisation of `users` via `user_id` UNIQUE. Carries company-facing attributes (`company_name`, `company_website`, `company_description`). ON DELETE CASCADE from `users` ensures profile rows are removed when a user is removed.

job_vacancies

The 24-column central table. The first column is `employer_id` – this is what enforces ownership: the employer dashboard only ever queries `WHERE employer_id = :me`. The 14 foreign keys (title, category, employment type, industry, level, country, city, district, work arrangement, salary range, salary type, degree level, experience level, plus employer) replace what would otherwise be 14 free-text columns. Five TEXT columns hold genuinely unstructured employer prose (responsibilities, required qualifications, preferred skills, benefits, additional notes); these are the only intentionally free-form fields.

job_vacancy_skills

A junction table with a composite primary key (`job_vacancy_id`, `skill_id`). Adding the same skill twice to the same vacancy is therefore impossible at the database level. The third column `proficiency_level_id` is conceptually an attribute of the link, not of the skill or the vacancy, which is the textbook reason for using a junction with payload columns. ON DELETE CASCADE on `job_vacancy_id` keeps the table clean if a vacancy is hard-deleted; in practice the application uses soft delete via `status`.



Lookup tables

Each lookup table has the same shape: `id` (surrogate key), `name` or `label` (UNIQUE), and `status`. The `status` column lets administrators retire stale entries (e.g. an obsolete job title) without breaking historical FKs from `job_vacancies`. `salary_ranges` is the only “richer” lookup – it additionally stores `min_salary`, `max_salary` and `currency`.

Location tables

`cities` is unique on (`country_id`, `name`) so two countries can both have a “Springfield”. Similarly `districts` is unique on (`city_id`, `name`). The chain enforces referential integrity, so a vacancy cannot point to a district that does not belong to the selected city.

7 Normalization Analysis

This section walks through First, Second, Third and Boyce–Codd normal forms with reference to the actual schema. Section 5 already justified the design at a higher level; the goal here is to demonstrate, table by table, that the schema is in BCNF.

7.1 First Normal Form (1NF)

A relation is in 1NF when every attribute is atomic (no repeating groups, no list-valued attributes). The schema satisfies 1NF because:

- No column stores a list. The required-skills set, which is the natural candidate for a list, lives in its own table `job_vacancy_skills`.
- No column packs structured data into JSON or comma-separated text. This is mandated by the specification.
- Location is decomposed into `country_id`, `city_id` and optional `district_id` rather than a single string like “Hanoi, Vietnam”.
- TEXT columns (`responsibilities`, `benefits`, etc.) are textually long but are still single, atomic values that are not joined or sorted on; they are properly atomic for the purposes of 1NF.

7.2 Second Normal Form (2NF)

2NF additionally requires that every non-key attribute depend on the *whole* primary key. Most tables use a single-column surrogate `id`, so the question only arises for the composite-key table `job_vacancy_skills`, whose primary key is (`job_vacancy_id`, `skill_id`).

The only non-key column is `proficiency_level_id`. The minimum proficiency the employer requires depends jointly on *which vacancy* and *which skill*, never on one without the other. Hence `proficiency_level_id` is fully functionally dependent on the composite key. The table is in 2NF.

7.3 Third Normal Form (3NF)

3NF additionally bans transitive dependencies. The notable risk is in `job_vacancies` where, on a denormalised version, a designer might be tempted to also store `country_name`, `city_name` or `salary_range_label` for read convenience. This is rejected here:

- Every redundant attribute that could be derived from a foreign key has been moved to the lookup table.
- In particular, `salary_ranges.min_salary` and `salary_ranges.currency` live only in the lookup table and are read via JOIN at search time. There is no `job_vacancies.salary_min` column.
- The location chain enforces that the `country` of a city, and the `city` of a district, is read via the parent FK rather than duplicated.

The only “transitive-looking” dependency in `job_vacancies` is `job_vacancy.country_id` versus `job_vacancy.city_id`: a city already belongs to a country, so storing `country_id` on the vacancy seems redundant. This is a deliberate denormalisation: the search controller filters by country in many cases without needing to join through cities, and the slight write-time redundancy is justified by the read-time simplification. The application validates that `cities.country_id` matches `job_vacancies.country_id` on insert, which keeps the two columns consistent.



7.4 Boyce–Codd Normal Form (BCNF)

For every non-trivial functional dependency $X \rightarrow Y$, X must be a superkey. The lookup tables trivially satisfy this; the central table `job_vacancies` also satisfies it because every non-key column depends on `id` (the only candidate key) and on nothing else. The junction `job_vacancy_skills` satisfies BCNF as discussed above. The location tables satisfy BCNF because their unique business keys (`(country_id,name)` for cities, `(city_id,name)` for districts) are the only superkeys aside from `id`.

7.5 Why a Junction Table for Skills?

The specification offers two implicit options: storing skills as a list (forbidden) or as a junction table (recommended). The junction table wins for several practical reasons:

- It permits per-relationship attributes – in this project, the minimum proficiency.
- It enforces uniqueness of `(job_vacancy_id, skill_id)` via the composite PK.
- It supports efficient queries such as “find all jobs requiring *Python*” as a single equality on `skill_id`, with an index supporting `ORDER BY job_vacancy_id` naturally given the composite PK.
- It does not lock the schema to a maximum cardinality; the per-job cap of 5 is enforced in PHP and can be relaxed without a migration.

7.6 Indexes

InnoDB automatically creates an index on every primary key and on every column declared `UNIQUE`, and an index on every foreign key. Because every selectable filter is a foreign key, the system has free read indexes for `country_id`, `city_id`, `district_id`, `employment_type_id`, `job_level_id`, `salary_range_id`, `work_arrangement_id`, `job_category_id`, `job_title_id`, `employer_id`, etc. Keyword search uses `LIKE '%kw%'` on `job_titles.name` and on the description text columns; for a project of this size MySQL's standard indexes are sufficient and a full-text index would be over-engineering.

8 Job Search Implementation

8.1 Filter Vocabulary

Section 4.2.1 of the specification lists eight filter dimensions. Each one is implemented as a single optional WHERE clause in `JobVacancy::search()`:

Filter	Predicate
Keyword	<code>(jt.name LIKE :kw OR jv.responsibilities LIKE :kw OR jv.required_qualifications LIKE :kw)</code>
Job category	<code>jv.job_category_id = :cat</code>
Country	<code>jv.country_id = :country</code>
City	<code>jv.city_id = :city</code>
Required skill	<code>EXISTS (SELECT 1 FROM job_vacancy_skills jvs WHERE jvs.job_vacancy_id = jv.id AND jvs.skill_id = :skill)</code>
Employment type	<code>jv.employment_type_id = :emp_type</code>
Job level	<code>jv.job_level_id = :level</code>
Salary range	<code>jv.salary_range_id = :salary</code>
Work arrangement	<code>jv.work_arrangement_id = :arrangement</code>

Crucially, the public search always also adds `jv.status = 'active'`. `removed` and `inactive` vacancies never reach a job seeker.

8.2 AND-Logic Combination

The specification (§4.2.2) requires that filters combine with AND-logic. The controller does *not* build a string by concatenation; instead it appends predicates and parameters into two parallel arrays, then joins them with AND once:

Listing 1: Filter assembly inside `PublicJobController::index()`.

```
1 $where = ["jv.status = 'active'"];
2 $params = [];
3
4 if (!empty($q['keyword'])) {
5     $where[] = "(jt.name LIKE :kw
6                 OR jv.responsibilities LIKE :kw
```

```
7         OR jv.required_qualifications LIKE :kw)";
8     $params[':kw']      = '%' . $q['keyword'] . '%';
9 }
10 if (!empty($q['country_id'])) {
11     $where[]            = "jv.country_id = :country";
12     $params[':country'] = (int) $q['country_id'];
13 }
14 // ... one block per filter ...
15
16 $sql = "SELECT jv.*, jt.name AS job_title, ...
17         FROM job_vacancies jv
18         JOIN job_titles jt   ON jv.job_title_id = jt.id
19         JOIN ...
20         WHERE " . implode(' AND ', $where) . "
21         ORDER BY $orderBy
22         LIMIT :off, :size";
```

This approach has three advantages: (i) every value reaches the database via a bound parameter (no string concatenation, no SQL injection); (ii) any subset of filters can be active simultaneously without combinatorial code; (iii) the AND-logic is structurally guaranteed by the `implode(' AND ', $where)`.

8.3 Sorting

Three sort keys are exposed (recently posted, salary, alphabetical). To prevent `ORDER BY` injection, the controller never accepts the raw user value – instead it maps a small set of allowed keys onto a hard-coded SQL fragment.

Listing 2: Whitelisted `ORDER BY` resolution.

```
1 $sortMap = [
2     'recent'      => 'jv.created_at DESC',
3     'salary_asc'  => 'sr.min_salary ASC',
4     'salary_desc' => 'sr.min_salary DESC',
5     'title_asc'   => 'jt.name ASC',
6 ];
7 $orderBy = $sortMap[$q['sort']] ?? 'recent' ?? 'jv.created_at DESC';
```

The salary sort uses `salary_ranges.min_salary` – the dedicated numeric column added precisely so that “ascending” and “descending” produce a meaningful order even when the label is a textual “1000–1500 USD”.

8.4 Job Detail View

The detail page issues a single query joining `job_vacancies` with all relevant lookup tables and a sub-aggregation of skills, so the view never makes additional database calls. The skills are returned as a comma-separated string built by `GROUP_CONCAT(skill_name, '(', proficiency_name, ')') ORDER BY skill_name`.

8.5 Performance Notes

For the dataset sizes targeted by the assignment (a few hundred to a few thousand vacancies, low double-digit lookup rows), the schema is fast without any extra tuning: every filter is on an indexed FK, the keyword `LIKE` is on a small-width `VARCHAR` column, the `EXISTS` subquery for skills uses the composite PK index. If the system were to grow to tens of thousands of vacancies the natural extensions would be a full-text index on the description columns and an extra index on `(status, created_at)` for the public list.

9 Security and Validation

The system follows a defence-in-depth approach. The four pillars are role-based access control, ownership protection, server-side validation, and parameterised queries. Each pillar is briefly described below.

9.1 Authentication

Passwords are hashed with PHP's `password_hash($plain, PASSWORD_DEFAULT)`, which currently maps to `bcrypt` with a per-row salt. Verification uses `password_verify`. Plaintext passwords are never written to disk and never logged. The session id is regenerated on successful login (`session_regenerate_id(true)`) to mitigate session fixation.

9.2 Role-Based Access Control

The `Auth` core class provides three guard methods:

- `Auth::user()` – returns the current session user or `null`.
- `Auth::requireLogin()` – redirects unauthenticated visitors to login.

- `Auth::requireRole($role)` – redirects users whose role is not `$role` to the public homepage.

Every employer action invokes `requireRole('employer')` at the top of the controller method, every admin action invokes `requireRole('admin')`. Public job search and detail are reachable without login.

9.3 Ownership Protection

A correct role guard is necessary but not sufficient: a logged-in employer must not be able to edit *another* employer's vacancy. The controller therefore performs a row-level check before mutating data, using a small helper:

Listing 3: Row-level ownership check.

```
1 private function ensureOwned(int $vacancyId): array
2 {
3     $job = $this->jobModel->find($vacancyId);
4     if (!$job
5         || $job['employer_id'] !== Auth::user()['employer_id']) {
6         $this->forbidden(); // sets HTTP 403, redirects to dashboard
7         exit;
8     }
9     return $job;
10 }
```

The same idea protects `?page=employer_job_edit&id=...` from URL-tampering attacks: changing the `id` parameter to another employer's vacancy results in 403 with no information leaked about whether that `id` exists.

9.4 Server-Side Validation

Client-side validation (`HTML required`, `type="number"`, `maxlength`) is treated as a UX nicety only. The authoritative validation is server-side and lives in `Validator`. For each form, the controller:

1. Type-coerces every numeric input.
2. Re-fetches every selected lookup id from the database. If `job_titles` does not contain the submitted `job_title_id` (or the row is `inactive`), the form is rejected with a per-field error.



3. Confirms structural integrity in the location chain (the chosen `city_id` must belong to the chosen `country_id`; `district_id` – if present – must belong to the city).
4. Confirms the skills cap (≤ 5 skills) and that every skill id is unique within the submission.
5. Accumulates errors into an associative array; the form is re-rendered with the user's previous values when any error is present.

9.5 Database Hardening

- **Prepared statements everywhere.** Every `Database::pdo()->prepare()` uses bound parameters; SQL identifiers (sort columns) are resolved through a whitelist (Section 8).
- **Strict character set.** The schema uses `utf8mb4 / utf8mb4_unicode_ci` so that all Unicode strings (including emoji and Vietnamese diacritics) are stored consistently.
- **Foreign key integrity.** Every reference column has an `ON DELETE` or `ON UPDATE` policy. Cascading deletes are reserved for the user/profile and vacancy/skill links; lookup deletes are restricted to avoid orphan vacancies.
- **Soft delete for vacancies.** The admin moderation pathway uses `status='removed'` rather than `DELETE`, preserving historical data for audit.

9.6 Output Encoding

All view templates pass dynamic data through a small `e()` helper that calls `htmlspecialchars($value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8')`. This prevents reflected and stored XSS in fields such as `company_name` and `benefits`. URLs that include a vacancy id use `(int)` casts before being placed into anchors.

9.7 CSRF Considerations

The current implementation keeps state-changing actions behind authenticated POST workflows, server-side validation, role guards, ownership checks, and prepared statements. A complete per-form CSRF-token mechanism is identified as a recommended hardening step for future production deployment.



10 User Interface and Experience

10.1 Layout System

The system uses a single shared layout (`resources/views/layout/app.php`) that wraps every page with a header, navigation and footer. Pages are slotted in through a content variable. A second layout (`layout/dashboard.php`) is used inside employer and admin areas to surface a side-navigation and the user's name and role. CSS is kept in two separate stylesheets – one for public pages, one for the dashboards – to keep the public bundle small.

10.2 Responsive Design

The layout uses a fluid container with a viewport meta tag, and CSS Grid / Flexbox for the major regions. The breakpoints are deliberately simple: a single major breakpoint at 768 px collapses the side navigation into a hamburger menu and makes filter selects full-width. Forms use `display: grid; grid-template-columns: repeat(2, 1fr);` on desktop and `repeat(1, 1fr);` on mobile.

10.3 Dynamic Forms

Three pieces of the employer form behave dynamically:

- **Cascading location selects.** When the country changes, JavaScript fetches the cities for that country (`?page=cities&country_id=...`, JSON response) and rebuilds the city select. The same pattern populates districts when the city changes.
- **Required-skills picker.** The skills row is a small component that lets the employer add a skill + minimum proficiency, then add another up to five rows. JavaScript prevents adding a sixth row, and prevents adding the same skill twice. Each row's hidden inputs are renamed to `skills[0][skill_id]`, `skills[0][proficiency_id]`, etc., so PHP receives a clean associative array.
- **Salary range hint.** When the employer picks a salary range, the helper text below the field is updated with the corresponding `min_salary - max_salary currency` string fetched from the lookup row.

10.4 Accessibility

Forms use `<label for="...">` pairings, every interactive control is reachable by keyboard, and visible focus styles are kept (the project never sets `outline: none`). Error messages are placed adjacent to the offending field and use a colour combination that meets WCAG AA contrast.

10.5 Internationalisation

A lightweight translation helper – `app/Helpers/lang.php` – looks up keys in an associative array and falls back to the key itself if a translation is missing. The dashboard navigation provides an English/Vietnamese language switch through `LanguageController`, which sets a session variable and reloads the current page.

11 Testing and Demo

11.1 Test Strategy

Given the time scope of the assignment the project relies on *manual integration testing* rather than an automated test suite. Tests are organised as a checklist (`docs/test-checklist.md`) covering the workflows below.

11.2 Manual Test Checklist

1. **Authentication.** Register a new employer; log out; log back in. Try logging in with an unknown email; check that the error is generic (no information leak about whether the email exists).
2. **Role guard.** As a job-seeker, try to open `?page=employer_dashboard`; expect redirect to login or homepage with no information leak.
3. **Vacancy creation – happy path.** Fill every field, pick five skills, save with status active. Open the public homepage in a private window; the vacancy is visible.
4. **Vacancy creation – validation.** Submit with an empty title; expect a per-field error. Submit with a non-existent `job_title_id` hand-crafted in DevTools; expect rejection with the same per-field error.



5. **Skill cap.** Try to add six skills via DevTools; expect rejection.
6. **Location cascade.** Pick a country, then change country; expect the city dropdown to clear. Manually post a city id from another country; expect rejection.
7. **Edit ownership.** Open the edit URL of another employer's vacancy; expect 403.
8. **Activate / deactivate.** Toggle status; reload the public page; the vacancy disappears or reappears as expected.
9. **Search filters.** Apply each filter individually, then several together; verify AND-logic visually.
10. **Sort.** Switch between "recent", "salary asc", "salary desc" and "title asc"; verify ordering.
11. **Detail view.** Open the detail page; confirm every field renders, including the comma-separated required skills with proficiency in parentheses.
12. **Admin removal.** As admin, set the vacancy to **removed**; confirm the vacancy is gone from the public list, kept in the database.
13. **Reference data CRUD.** Add a new **industry**, set its status to inactive, confirm it does not appear in the employer form select.

11.3 Demo Script

A short demo script (`docs/demo-script.md`) is provided for the recorded walkthrough; the recommended order is: (i) employer login, (ii) create vacancy, (iii) public search and detail, (iv) admin moderation.

11.4 Known Edge Cases

- **Empty lookup tables.** If an administrator deactivates every job category, the employer form select is empty and the form refuses to submit; the controller surfaces a clear error rather than crashing.
- **Inactive lookup row referenced by an existing vacancy.** The admin can deactivate a category that an old vacancy still references. The vacancy remains valid (the FK still resolves) but the category is no longer offered to new employers.



- **Soft-removed vacancy.** The skill links survive (`ON DELETE CASCADE` only fires on a hard delete) and become reachable again if the admin reverts the status to **active**.

12 Limitations and Future Work

12.1 Deliberate Out-of-Scope Items

The features below are not implemented because the assignment specification excludes them, not because of technical difficulty:

- No CV creation and no CV search. Assignment 1 covers those.
- No job application workflow – the system is read-only for job seekers.
- No matching or recommendation algorithm.
- No email notification, chat or payment workflow.

12.2 Limitations of the Current Implementation

Even within scope, a few areas are intentionally simple:

- **Search relevance.** The keyword search uses naive substring matching with `LIKE '%kw%'` on the title and two description columns. A relevance ranking would require either a full-text index or an external search engine.
- **Pagination.** The public list paginates via `LIMIT/OFFSET`. For very large data sets this is suboptimal; cursor-based pagination would scale better.
- **Skill cap enforcement.** The maximum of five required skills per vacancy is enforced in PHP. Moving the cap into the database (e.g. via a stored procedure or an application-checked invariant) would be more robust if other writers were added.
- **Audit log.** Status changes are not recorded with a who/when timestamp; only the latest `updated_at` is kept. An `audit_events` table would be a small but useful extension.

12.3 Future Work

The schema is designed to extend cleanly:

- **Job applications.** Add `applications(id, job_vacancy_id, job_seeker_id, status, created_at)`; everything else stays the same.
- **Saved searches and alerts.** Add `saved_searches` keyed by user with serialised filter payloads.
- **Localisation of lookup labels.** Either add a `name_vi` column to each lookup table, or model translations as a separate `lookup_translations(table_name, row_id, locale, label)` table.
- **Full-text search.** Add a MySQL full-text index on `(jt.name, jv.responsibilities, jv.required_qualifications)` or migrate to a dedicated search engine (Meilisearch / Elasticsearch).
- **Image uploads for company logos.** Reserved `storage/` directory and a single column on `employer_profiles`.

13 Conclusion

The VietTalent system delivers the two complementary halves of the recruitment workflow required by the specification: a structured employer-facing vacancy management dashboard, and a public job seeker search experience.

The schema – 19 tables, 21 referential relationships, one many-to-many junction with a payload column, a three-level location hierarchy and a salary range table that captures both a human label and a numeric column for sorting – has been driven by three principles. First, every selectable attribute is a foreign key into a reference table, never free text. Second, location is decomposed instead of stored as a string. Third, soft delete via `status` preserves history.

These principles produce three concrete benefits visible to the user: consistent select option labels everywhere in the UI, multi-criteria AND-logic search that simply concatenates equality predicates over indexed columns, and reversible moderation actions for the administrator. They also keep the backend code small: every controller is a thin layer over



a model whose only responsibility is to issue parameterised queries. The result is a clean, normalised, BCNF-compliant data model that is straightforward to read, easy to extend and well aligned with the assessment criteria.

1 Appendix: SQL Schema File

The complete DDL is included in the project submission at `database/schema.sql`. A copy is also kept in `listings/schema.sql` inside this LaTeX package so that the report source can be compiled and reviewed independently. The DDL is not printed in full inside the report body to keep the PDF readable.

2 Appendix: Default Demo Accounts

The seed script (`database/seed.sql`) creates three accounts for the demo:

Role	Email	Password
Admin	<code>admin@viettalent.local</code>	<code>password</code>
Employer	<code>employer@viettalent.local</code>	<code>password</code>
Job seeker	<code>jobseeker@viettalent.local</code>	<code>password</code>



3 Appendix: Main Routes

Route	Description
/public/	Homepage with the latest active jobs
?page=login / ?page=register	Authentication
?page=jobs	Public search and listing
?page=job_detail&id={id}	Public detail view
?page=employer_dashboard	Employer landing
?page=employer_jobs	Employer's own vacancy list
?page=employer_job_create	Vacancy creation form
?page=admin_dashboard	Administrator landing
?page=admin_jobs	Administrator vacancy moderation
?page=admin_lookup&type=skills	Lookup-table CRUD
?page=admin_countries / _cities / _districts	Location-table CRUD

4 Appendix: Website Setup and Run Instructions

1. Copy the project folder to C:
xampp
htdocs
VietTalent_JobVacancyManagement.
2. Start Apache and MySQL from the XAMPP Control Panel.
3. Create the database viettalent_job_vacancy_db in phpMyAdmin.
4. Import database/schema.sql.
5. Import database/seed.sql.
6. Check that config/database.php matches the local MySQL username, password and database name.
7. Open `http://localhost/VietTalent_JobVacancyManagement/public/` in a browser.



5 Appendix: Build Instructions

The LaTeX source can be compiled with XeLaTeX:

```
cd viettalent_report_template  
xelatex -interaction=nonstopmode report.tex  
xelatex -interaction=nonstopmode report.tex
```